

ЛР7 (3 пары / 6 часов)

«Импорт/экспорт JSON + резервная копия tasks.json (backup) + защита от ошибок файла»

Что уже есть

- Задачи сохраняются в `data/tasks.json`
- CRUD работает
- Поиск/фильтр/сортировка работают

Что делаем в ЛР7

Добавим **настоящие "системные" функции** (очень жизненно):

1. **Backup**: сделать резервную копию `tasks.json` в папку `backups`
 2. **Export**: сохранить текущий список задач в файл `exports/tasks_export_...json`
 3. **Import**: загрузить задачи из выбранного JSON-файла (по пути) и заменить текущие
-

0) Итог ЛР7 (что должно работать)

✓ В меню появились пункты:

- 9) Сделать резервную копию (backup)
- 10) Экспорт в файл (export)
- 11) Импорт из файла (import)

✓ При backup создаётся файл вида:

`backups/tasks_YYYY-MM-DD_HH-mm-ss.json`

✓ При export создаётся файл вида:

`exports/tasks_export_YYYY-MM-DD_HH-mm-ss.json`

✓ При import программа:

- просит путь к JSON
 - проверяет "файл существует?"
 - если JSON неверный → пишет "ошибка импорта" и **не падает**
 - если всё ок → **заменяет список** и сохраняет в `data/tasks.json`
-

1) Теория (развернуто, но только важное)

1.1. Зачем backup вообще нужен

Если файл данных испортился (случайно удалили/сломали JSON), резервная копия спасает.

Правило: перед важными изменениями или перед импортом — сделать backup.

1.2. Экспорт и импорт — чем отличаются от "обычного сохранения"

- Обычное сохранение всегда в **один файл**: `data/tasks.json`
- Экспорт — сохранить "копию" в **отдельный файл**, например для отправки преподавателю/другу
- Импорт — загрузить **из другого файла** и заменить текущие задачи

1.3. Почему импорт опасен

Импорт может:

- стереть текущие задачи
- принести "битый" JSON

Поэтому мы делаем:

- подтверждение "точно импортировать?"
- backup перед импортом (минимум как рекомендация)

1.4. Базовая защита от ошибок файла

Мы всегда должны уметь сказать:

- "Файл не найден"
 - "JSON поврежден"
 - "Не хватает прав доступа"
- и не падать.
-

2) Практика. Пара 1: подготовка Core — метод ReplaceAll (замена списка задач)

Импорт должен "перезагрузить" задачи в сервис. Для этого добавим метод в `TaskService`.

Шаг 1. Открыть TaskService

Visual Studio → `TaskTracker.Core` → `Services` → `TaskService.cs`

Шаг 2. Добавить метод ReplaceAll

Внутри класса `TaskService` (в самый низ, но внутри фигурных скобок класса) добавьте:

```
public void ReplaceAll(List<TaskItem> newTasks)
{
    newTasks ??= new List<TaskItem>();

    _tasks.Clear();
    _tasks.AddRange(newTasks);

    // Пересчитать следующий Id
    _nextId = _tasks.Count == 0 ? 1 : _tasks.Max(t => t.Id) +
1;
}
```

⚠ Если подсвечивается `AddRange` или `Max` — убедитесь, что сверху есть:

```
using System.Linq;
```

Шаг 3. Собрать проект

Build → Build Solution → должно быть **Build succeeded**.

Шаг 4. Коммит №1

Сообщение:

- `feat: add ReplaceAll method to TaskService for import`

Commit All → Push.

3) Практика. Пара 2: добавляем "службы" в Storage — Backup и Export/Import

Мы сделаем простую службу для резервного копирования и экспорта/импорта.

Шаг 5. Создать файл BackupService.cs

В проекте `TaskTracker.Storage`:

1. Solution Explorer → `TaskTracker.Storage`
2. Правой кнопкой по папке `Services`
3. Add → Class...
4. Название: `BackupService.cs`
5. Add

Шаг 6. Вставить код BackupService

Откройте `BackupService.cs` и вставьте:

```
namespace TaskTracker.Storage.Services;

public static class BackupService
{
    public static string CreateBackup(string sourceFilePath,
    string backupsFolder)
    {
        if (!File.Exists(sourceFilePath))
            throw new FileNotFoundException("Файл данных не найден.", sourceFilePath);

        Directory.CreateDirectory(backupsFolder);

        var fileName = $"tasks_{DateTime.Now:yyyy-MM-dd_HH-mm-ss}.json";
        var destPath = Path.Combine(backupsFolder, fileName);

        File.Copy(sourceFilePath, destPath, overwrite: false);

        return destPath; // путь к созданному бэкапу
    }
}
```

Шаг 7. Экспорт/Импорт через JsonTaskStorage (он уже есть)

Мы не создаём новый класс — проще:

- для экспорта: создаём новый `JsonTaskStorage(exportPath)` и делаем `Save(tasks)`
- для импорта: создаём `JsonTaskStorage(importPath)` и делаем `Load()`

Это проще для новичков.

Шаг 8. Сборка

Build → Build Solution → Build succeeded.

Шаг 9. Коммит №2

Сообщение:

- `feat: add BackupService for tasks.json backups`

Commit All → Push.

4) Практика. Пара 3: добавляем пункты меню 9–11 в App + отчёт

Шаг 10. Открыть Program.cs

`TaskTracker.App → Program.cs`

Убедитесь, что вверху есть:

```
using TaskTracker.Storage.Services;
```

(он уже был для JsonTaskStorage, но BackupService тоже в этом namespace)

Шаг 11. Подготовить пути (backups и exports)

Найдите место, где вы задаёте `dataFilePath` (ЛР4). Сразу после него добавьте:

```
var backupsFolder = Path.Combine(AppContext.BaseDirectory, "backups");  
var exportsFolder = Path.Combine(AppContext.BaseDirectory, "exports");
```

Шаг 12. Добавить пункты меню

В вашем выводе меню добавьте:

```
Console.WriteLine("9) Сделать резервную копию (backup)");  
Console.WriteLine("10) Экспорт в файл (export)");  
Console.WriteLine("11) Импорт из файла (import)");
```

Шаг 13. Реализовать пункт 9: backup

Добавьте в `while(true)` блок:

```
if (input == "9")  
{  
    try  
    {  
        // Сохраним текущие данные на всякий случай  
        storage.Save(service.GetAll());  
  
        var backupPath = BackupService.CreateBackup(dataFilePath, backupsFolder);  
        Console.WriteLine("Бэкап создан: " + backupPath);  
    }  
    catch (Exception ex)  
    {  
        Console.WriteLine("Ошибка backup: " + ex.Message);  
    }  
  
    continue;  
}
```

Шаг 14. Реализовать пункт 10: export

Добавьте блок:

```
if (input == "10")  
{  
    try
```

```

{
    Directory.CreateDirectory(exportsFolder);

    var exportFile = Path.Combine(
        exportsFolder,
        $"tasks_export_{DateTime.Now:yyyy-MM-dd_HH-mm-ss}.json"
    );

    var exportStorage = new JsonTaskStorage(exportFile);
    exportStorage.Save(service.GetAll());

    Console.WriteLine("Экспорт выполнен: " + exportFile);
}
catch (Exception ex)
{
    Console.WriteLine("Ошибка export: " + ex.Message);
}

continue;
}

```

Шаг 15. Реализовать пункт 11: import (замена задач)

Добавьте блок:

```

if (input == "11")
{
    Console.WriteLine("Импорт заменит текущий список задач!");
    Console.Write("Введите путь к JSON-файлу для импорта: ");
    var importPath = (Console.ReadLine() ?? "").Trim();

    if (string.IsNullOrEmpty(importPath))
    {

```



```

        Console.WriteLine("Ошибка: путь пустой.");
        continue;
    }

    if (!File.Exists(importPath))
    {
        Console.WriteLine("Ошибка: файл не найден: " + import
Path);
        continue;
    }

    Console.Write("Сделать backup перед импортом? (y/n): ");
    var backupAnswer = (Console.ReadLine() ?? "").Trim().ToLower();

    if (backupAnswer == "y")
    {
        try
        {
            storage.Save(service.GetAll());
            var backupPath = BackupService.CreateBackup(dataF
ilePath, backupsFolder);
            Console.WriteLine("Backup создан: " + backupPat
h);
        }
        catch (Exception ex)
        {
            Console.WriteLine("Не удалось сделать backup: " +
ex.Message);
            Console.WriteLine("Импорт отменён (без backup опа
сно).");
            continue;
        }
    }

    Console.Write("Точно импортировать и заменить задачи? (y/

```

```

n): ");
    var sure = (Console.ReadLine() ?? "").Trim().ToLower();
    if (sure != "y")
    {
        Console.WriteLine("Импорт отменён.");
        continue;
    }

    try
    {
        var importStorage = new JsonTaskStorage(importPath);
        var importedTasks = importStorage.Load();

        // Заменяем задачи в сервисе
        service.ReplaceAll(importedTasks);

        // Сохраняем в основной файл data/tasks.json
        storage.Save(service.GetAll());

        Console.WriteLine("Импорт выполнен. Загружено задач:
" + importedTasks.Count);
    }
    catch (Exception ex)
    {
        Console.WriteLine("Ошибка import: " + ex.Message);
    }

    continue;
}

```

Шаг 16. Проверка (F5) — обязательные сценарии

Сценарий А: бэкап работает

1. Запустить (F5)

2. Добавить 1–2 задачи
3. Нажать **9**
4. Проверить: появилось сообщение “Бэкап создан...”
5. Открыть папку **backups** (рядом с приложением) — там есть файл **tasks_....json**

Сценарий В: export работает

1. Нажать **10**
2. Проверить папку **exports** — там есть **tasks_export_....json**

Сценарий С: import работает

1. Очистите список вручную (например удалите задачи) **или** создайте отдельный экспорт
2. Нажмите **11**
3. Вставьте путь к файлу из **exports** (можно скопировать из сообщения экспорт)
4. Согласитесь на backup (y)
5. Подтвердите импорт (y)
6. Нажмите **2** (список) — задачи должны стать такими, как в импортируемом файле

Сценарий D: ошибки не ломают программу

1. **11** → путь **C:\no_such_file.json**
→ должно написать “файл не найден”, программа не закрывается
2. Создайте “битый” json-файл (например, напишите туда **123**) и попробуйте импорт
→ должна быть ошибка, но без падения

Шаг 17. Отчёт **docs/LR7_Report.md**

Создайте файл `docs/LR7_Report.md`:

```
# Отчёт ЛР7 – TaskTracker
```

```
## Что сделано
```

- Добавлены функции:
 - Backup tasks.json в папку backups
 - Export задач в отдельный файл в папке exports
 - Import задач из выбранного JSON-файла (замена списка)
- Добавлен метод ReplaceAll в TaskService для импорта
- Добавлена обработка ошибок (файл не найден, неверный путь)

```
## Как проверить
```

- 1) Запустить программу (F5)
- 2) Добавить 2 задачи
- 3) Нажать 9 -> создать backup
- 4) Нажать 10 -> экспорт, проверить файл в exports
- 5) Нажать 11 -> импорт из экспортированного файла, подтвердит
ь у
- 6) Показать список (2) -> задачи восстановились/заменялись

Шаг 18. Коммит №3

Сообщение:

- `feat: add backup/export/import commands and LR7 report`

Commit All → Push.

5) Что сдавать (чек-лист)

✓ ссылка на GitHub

✓ `docs/LR7_Report.md`

✓ видео/скрин 1–2 минуты:

- экспорт → импорт → показ списка

- backup создан

✓ минимум 3 коммита за ЛР7

6) Частые проблемы (и что говорить детям)

“Я не понимаю какой путь вводить для импорта”

Самый простой способ:

1. Сначала сделай Export (пункт 10)
2. Программа покажет путь файла
3. Выдели путь мышкой в консоли → Ctrl+C
4. Вставь этот путь в import

“Backups/Exports не видно”

Папки появляются **рядом с папкой запуска** (где лежит exe).

Они обычно в `bin\Debug\net8.0\`.
